Assignment #3

Puneet Velidi

2026-02-20

Q1 – Boundary Value Problem (10 points)

We're given the BVP

$$(a(x)\,y')' = f(x), \quad 0 \le x \le 1, \quad y(0) = 1,\ y(1) = 2,$$

with $a(x) = 1 + x^2$ and $f(x) = 2 + 6x^2$.

(a) Uniqueness and exact solution

Uniqueness (energy method)

Assume two solutions y_1, y_2 exist and see what happens. Let $w = y_1 - y_2$. Then w satisfies:

$$(a(x)\,w')' = 0, \quad w(0) = 0,\ w(1) = 0.$$

Multiply both sides by w and integrate:

$$\int_0^1 w\,(a(x)\,w')'\,dx = 0.$$

Integrate by parts. The boundary term $[w\,a(x)\,w']_0^1$ vanishes since $w(0) = w(1) = 0$, so we're left with:

$$-\int_0^1 a(x)\,(w')^2\,dx = 0 \quad \Longrightarrow \quad \int_0^1 a(x)\,(w')^2\,dx = 0.$$

Now, $a(x) = 1 + x^2 > 0$ on $[0, 1]$ and $(w')^2 \ge 0$, so the only way this integral is zero is if $w' = 0$ everywhere:

$$w'(x) = 0 \ \ \forall\, x \quad \Longrightarrow \quad w \equiv \text{const} \quad \Longrightarrow \quad w \equiv 0 \quad (\text{since } w(0) = 0).$$

So $y_1 = y_2$. By the existence theorem for linear BVPs with smooth, positive $a(x)$, a solution exists — together with uniqueness, we get exactly one solution. ■

Checking that $y_e(x) = x^2 + 1$ is the solution

Just plug it in. We have $y_e' = 2x$, so:

$$a(x)\,y_e' = (1 + x^2)(2x) = 2x + 2x^3$$

$$\frac{d}{dx}\big[a(x)\,y_e'\big] = 2 + 6x^2 = f(x) \quad \checkmark$$

Boundary conditions: $y_e(0) = 0 + 1 = 1$ ✓, $y_e(1) = 1 + 1 = 2$ ✓.

Both check out. So $y_e(x) = x^2 + 1$ is the unique solution. ■

(b) Discretization using symmetric differencing

Set up a grid with n interior points: $x_i = ih$, $h = 1/(n+1)$, for $i = 0, \dots, n+1$. Define the half-point coefficients $a_{i\pm1/2} = a(x_i \pm h/2)$.

The idea is to approximate $\frac{d}{dx}[a(x)y']$ in two stages. First, centred differences at the half-points for the outer derivative:

$$\frac{d}{dx}[a(x)y']\bigg|_{x_i} \approx \frac{1}{h}\Big[a_{i+1/2}\, y'(x_i + h/2) - a_{i-1/2}\, y'(x_i - h/2)\Big]$$

Then approximate each y' at the half-points:

$$y'(x_i + h/2) \approx \frac{y_{i+1} - y_i}{h}, \qquad y'(x_i - h/2) \approx \frac{y_i - y_{i-1}}{h}$$

Putting it together, at each interior node $i = 1, \dots, n$:

$$\frac{1}{h^2}\Big[a_{i-1/2}\, y_{i-1} - (a_{i+1/2} + a_{i-1/2})\, y_i + a_{i+1/2}\, y_{i+1}\Big] = f(x_i)$$

This gives us a linear system $A_h Y = F$ with $Y = (y_1, \dots, y_n)^T$. The matrix A_h is tridiagonal:

$$(A_h)_{i,i} = -\frac{a_{i+1/2} + a_{i-1/2}}{h^2}, \quad (A_h)_{i,i-1} = \frac{a_{i-1/2}}{h^2}, \quad (A_h)_{i,i+1} = \frac{a_{i+1/2}}{h^2}$$

The boundary values $y(0) = 1$ and $y(1) = 2$ get absorbed into the first and last equations:

$$F_1 = f(x_1) - \frac{a_{1/2}}{h^2}\, y(0), \qquad F_n = f(x_n) - \frac{a_{n+1/2}}{h^2}\, y(1), \qquad F_i = f(x_i) \;\; (2 \le i \le n-1)$$

(c) Showing the truncation error is $O(h^2)$

We need to show that plugging the exact solution y_e into our difference formula leaves a residual that goes to zero like h^2. The truncation error at node x_i:

$$\tau_i = \frac{1}{h^2}\Big[a_{i-1/2}\, y_e(x_{i-1}) - (a_{i+1/2} + a_{i-1/2})\, y_e(x_i) + a_{i+1/2}\, y_e(x_{i+1})\Big] - f(x_i)$$

Expand everything in Taylor series around x_i:

$$y_e(x_i \pm h) = y_e \pm h y_e' + \frac{h^2}{2} y_e'' \pm \frac{h^3}{6} y_e''' + \frac{h^4}{24} y_e^{(4)} + \cdots$$

$$a_{i\pm1/2} = a(x_i) \pm \frac{h}{2} a'(x_i) + \frac{h^2}{8} a''(x_i) \pm \cdots$$

Multiply these out, substitute, and collect by powers of h:

- The $O(h^{-2})$ and $O(h^{-1})$ terms cancel out
- The $O(1)$ terms give $(a\, y_e')' = f(x_i)$, which cancels with $-f(x_i)$

- What's left starts at $O(h^2)$

$$\therefore \quad \tau_i = O(h^2), \qquad \|\tau_h\|_\infty = O(h^2) \qquad \blacksquare$$

(d) Showing A_h is non-singular

We want to show the system always has a unique solution. The approach is to show $-A_h$ is positive definite.

Take any nonzero $X = (x_1, \dots, x_n)^T$ and set $x_0 = x_{n+1} = 0$ (homogeneous boundary conditions). Doing a discrete summation by parts on $-X^T A_h X$:

$$-X^T A_h X = \frac{1}{h^2} \sum_{i=0}^{n} a_{i+1/2} \, (x_{i+1} - x_i)^2$$

Every term is non-negative since $a_{i+1/2} = 1 + (x_i + h/2)^2 > 0$. The sum can only be zero if $x_{i+1} = x_i$ for all i, but since $x_0 = 0$ that would force $X = 0$:

$$-X^T A_h X > 0 \;\; \forall \, X \neq 0 \quad \Longrightarrow \quad -A_h \text{ is positive definite} \quad \Longrightarrow \quad A_h \text{ is non-singular} \qquad \blacksquare$$

(e) Numerical experiments

Now let's solve the system for $h = 1/32, 1/64, 1/128$ and check that the error behaves like $O(h^2)$.

```
exact_sol <- function(x) x^2 + 1
a_fun     <- function(x) 1 + x^2
f_fun     <- function(x) 2 + 6 * x^2

solve_bvp <- function(n) {
  h  <- 1 / (n + 1)
  xi <- (1:n) * h

  ap <- a_fun(xi + h / 2)
  am <- a_fun(xi - h / 2)

  d_main  <- -(ap + am) / h^2
  d_upper <- ap[1:(n - 1)] / h^2
  d_lower <- am[2:n]       / h^2

  A <- diag(d_main)
  for (k in 1:(n - 1)) {
    A[k, k + 1] <- d_upper[k]
    A[k + 1, k] <- d_lower[k]
  }

  rhs    <- f_fun(xi)
  rhs[1] <- rhs[1] - am[1] / h^2 * 1
```

```
  rhs[n] <- rhs[n] - ap[n] / h^2 * 2

  Y <- solve(A, rhs)
  list(x = xi, Y = Y, h = h)
}

h_vals <- c(1/32, 1/64, 1/128)
n_vals <- round(1 / h_vals) - 1
errors <- numeric(length(h_vals))

for (k in seq_along(n_vals)) {
  sol       <- solve_bvp(n_vals[k])
  errors[k] <- max(abs(sol$Y - exact_sol(sol$x)))
}

rates <- log(errors[1:(length(errors) - 1)] / errors[2:length(errors)]) / log(2)

results <- data.frame(
  h   = h_vals,
  n   = n_vals,
  e_h = errors,
  r_h = c(NA, rates)
)

knitr::kable(results, digits = 10,
             col.names = c("h", "n", "e_h", "r_h"),
             caption = "Errors and convergence rates")
```

Table 1: Errors and convergence rates

h	n	e_h	r_h
0.0312500	31	4.61074e-05	NA
0.0156250	63	1.15259e-05	2.000117
0.0078125	127	2.88140e-06	2.000029

We get $r_h \approx 2$, confirming the scheme is second-order accurate. Each time we halve h, the error drops by roughly a factor of 4 — exactly what we'd expect from $e_h = O(h^2)$.

Q2 – Eigenvalues of a constant-coefficient tridiagonal matrix (5 points)

We have the $n \times n$ tridiagonal matrix with constant a on the diagonal, $c > 0$ on the sub-diagonal, and $b > 0$ on the super-diagonal:

$$A = \begin{pmatrix} a & b & & & \\ c & a & b & & \\ & c & \ddots & \ddots & \\ & & \ddots & a & b \\ & & & c & a \end{pmatrix}$$

Eigenvalues via difference-scheme method

Writing out $AV = \lambda V$ for the j-th row:

$$c\, v_{j-1} + a\, v_j + b\, v_{j+1} = \lambda\, v_j, \qquad j = 1, \dots, n$$

We treat this as a recurrence with boundary conditions $v_0 = 0$, $v_{n+1} = 0$. Try the ansatz:

$$v_j = \left(\frac{b}{c}\right)^{j/2} \sin(j\theta)$$

This gives $v_0 = \sin(0) = 0$, which is good. Plugging in and using the identity $\sin((j-1)\theta) + \sin((j+1)\theta) = 2\cos\theta \sin(j\theta)$:

$$2\sqrt{bc}\cos\theta\, \sin(j\theta) + a \sin(j\theta) = \lambda \sin(j\theta)$$

Since $\sin(j\theta) \neq 0$ for the eigenvectors we care about, we can divide through:

$$\lambda = a + 2\sqrt{bc}\, \cos\theta$$

For the other boundary condition $v_{n+1} = 0$, we need $\sin((n+1)\theta) = 0$, giving:

$$\theta_k = \frac{k\pi}{n+1}, \qquad k = 1, 2, \dots, n$$

Results

Eigenvalues:

$$\boxed{\lambda_k = a + 2\sqrt{bc}\, \cos\left(\frac{k\pi}{n+1}\right), \quad k = 1, \dots, n}$$

Eigenvectors: The k-th eigenvector has components:

$$v_j^{(k)} = \left(\frac{b}{c}\right)^{j/2} \sin\left(\frac{jk\pi}{n+1}\right), \quad j = 1, \dots, n$$

When is A non-singular? All eigenvalues must be nonzero:

$$a \neq -2\sqrt{bc}\,\cos\left(\frac{k\pi}{n+1}\right) \quad \forall\, k = 1, \ldots, n$$

Since $|\cos(k\pi/(n+1))| < 1$, a sufficient condition is $|a| > 2\sqrt{bc}$. ■

Q3 – Shooting method for a nonlinear BVP (10 points)

We want to solve:

$$v'' = x^2 + 3v + 10v^3, \quad 0 \le x \le 1, \quad v(0) = 0,\ v(1) = \beta$$

Setting up the IVP

Convert to a first-order system with $y_1 = v$, $y_2 = v'$:

$$y_1' = y_2, \qquad y_2' = x^2 + 3y_1 + 10y_1^3$$

The shooting idea: pick $v'(0) = \alpha$, solve the IVP, and adjust α until $v(1) = \beta$.

For Newton's method we also need the variational equation. Letting $w = \partial v / \partial \alpha$:

$$w'' = (3 + 30v^2)\, w, \qquad w(0) = 0,\ w'(0) = 1$$

The Newton update is:

$$\alpha_{k+1} = \alpha_k - \frac{v(1;\alpha_k) - \beta}{w(1;\alpha_k)}$$

```
ydotp_r <- function(t, y, parms) {
  dy1 <- y[2]
  dy2 <- 3 * y[1] + t^2 + 10 * y[1]^3
  dy3 <- y[4]
  dy4 <- (3 + 30 * y[1]^2) * y[3]
  list(c(dy1, dy2, dy3, dy4))
}

ydot_r <- function(t, y, parms) {
  dy1 <- y[2]
  dy2 <- 3 * y[1] + t^2 + 10 * y[1]^3
  list(c(dy1, dy2))
}
```

(a) Newton shooting – finding β_c

We use Newton with initial guess $\alpha = -1$ and try different values of β. Something goes wrong between $\beta = 3.5$ and $\beta = 4$, so we bisect on β to pin down where Newton stops converging.

```
newton_shooting <- function(beta, alpha0 = -1, tol = 1e-10, maxiter = 100) {
  s <- alpha0
  converged <- FALSE
  for (i in 1:maxiter) {
    sol <- tryCatch(
      ode(y = c(0, s, 0, 1), times = seq(0, 1, length.out = 500),
```

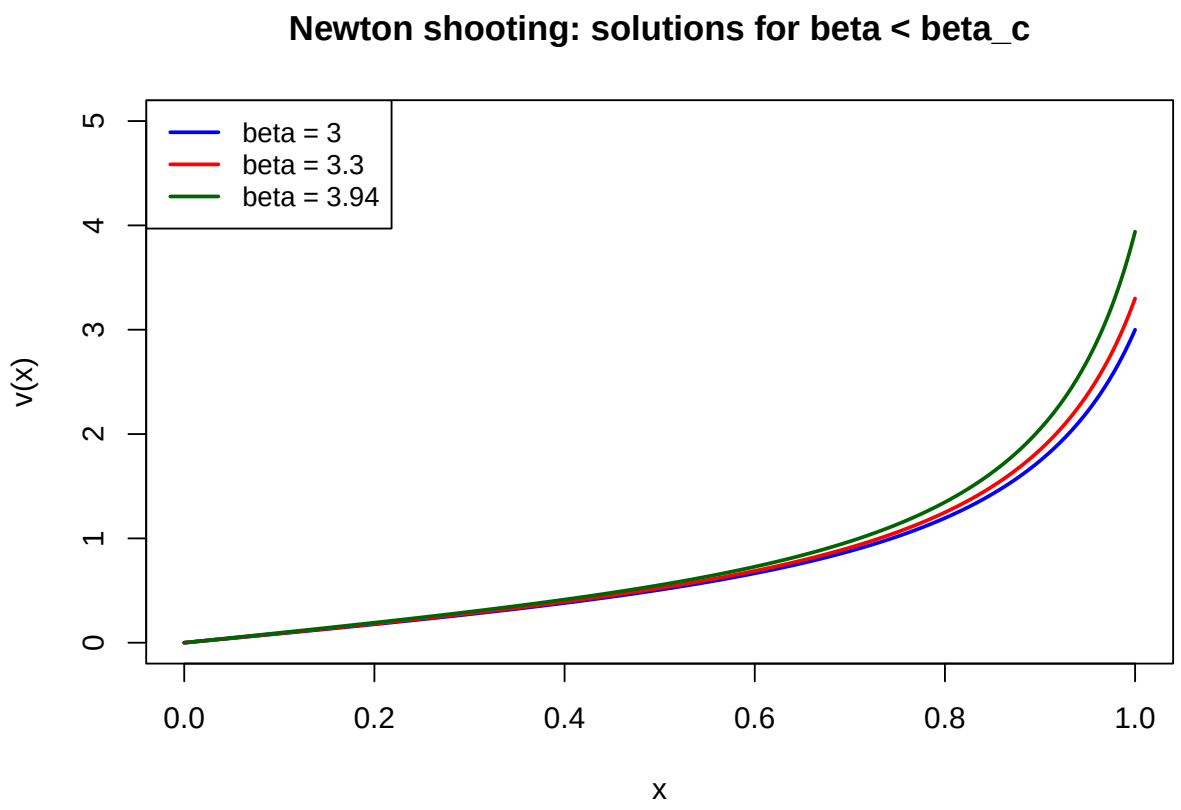
```
          func = ydotp_r, parms = NULL, method = "ode45"),
      error = function(e) NULL
    )
    if (is.null(sol) || any(!is.finite(sol[, 2]))) return(list(converged = FALSE))
    v1 <- tail(sol[, 2], 1)
    w1 <- tail(sol[, 4], 1)
    if (!is.finite(v1) || !is.finite(w1) || abs(w1) < 1e-15)
      return(list(converged = FALSE))
    s_new <- s - (v1 - beta) / w1
    if (!is.finite(s_new)) return(list(converged = FALSE))
    if (abs(s_new - s) < tol) { s <- s_new; converged <- TRUE; break }
    s <- s_new
  }
  if (!converged) return(list(converged = FALSE))
  sol_f <- ode(y = c(0, s), times = seq(0, 1, length.out = 500),
               func = ydot_r, parms = NULL, method = "ode45")
  list(x = sol_f[, 1], v = sol_f[, 2], alpha = s, converged = TRUE)
}

test_newton <- function(beta) {
  res <- newton_shooting(beta)
  res$converged && all(is.finite(res$v))
}

beta_lo <- 3.5; beta_hi <- 4.0
for (iter in 1:50) {
  beta_mid <- (beta_lo + beta_hi) / 2
  if (test_newton(beta_mid)) beta_lo <- beta_mid else beta_hi <- beta_mid
}
beta_c <- (beta_lo + beta_hi) / 2
cat(sprintf("Estimated beta_c = %.4f\n", beta_c))
```

```
## Estimated beta_c = 3.9580
```

```
betas_plot <- c(3.0, 3.3, round(beta_c - 0.02, 2))
plot(NULL, xlim = c(0, 1), ylim = c(0, 5),
     xlab = "x", ylab = "v(x)",
     main = "Newton shooting: solutions for beta < beta_c")
cols <- c("blue", "red", "darkgreen")
for (k in seq_along(betas_plot)) {
  res <- newton_shooting(betas_plot[k])
  if (res$converged) lines(res$x, res$v, col = cols[k], lwd = 2)
}
legend("topleft", legend = paste("beta =", betas_plot), col = cols, lwd = 2, cex = 0.9)
```



So Newton works fine for $\beta < \beta_c \approx 3.958$, but blows up past that. You might think the BVP just doesn't have a solution for $\beta > \beta_c$ — but as we'll see in part (b), that's not quite right.

(b) Bisection shooting and what's really going on

Now let's try bisection instead (using R's `uniroot`). Rather than using derivatives, bisection just brackets the root and narrows in.

```
shoot_residual <- function(alpha, beta) {
  sol <- tryCatch(
    ode(y = c(0, alpha), times = seq(0, 1, length.out = 2000),
        func = ydot_r, parms = NULL, method = "ode45"),
    error = function(e) NULL
  )
  if (is.null(sol) || any(!is.finite(sol[, 2]))) return(1e10 * sign(alpha))
  tail(sol[, 2], 1) - beta
}

bisection_shooting <- function(beta, lo = -10, hi = 10) {
  tryCatch({
    alpha_star <- uniroot(shoot_residual, interval = c(lo, hi),
                          beta = beta, tol = 1e-10, maxiter = 2000)$root
```

```
    sol <- ode(y = c(0, alpha_star), times = seq(0, 1, length.out = 500),
               func = ydot_r, parms = NULL, method = "ode45")
    list(x = sol[, 1], v = sol[, 2], alpha = alpha_star, converged = TRUE)
  }, error = function(e) list(converged = FALSE))
}

betas_test <- c(3.5, 3.8, 4.0, 4.5)
cat("Bisection shooting results:\n")
```

```
## Bisection shooting results:
```

```
for (b in betas_test) {
  res <- bisection_shooting(b)
  if (res$converged) {
    cat(sprintf("  beta = %.1f : converged, alpha = %.6f, v(1) = %.6f\n",
                b, res$alpha, tail(res$v, 1)))
  } else {
    cat(sprintf("  beta = %.1f : did not converge\n", b))
  }
}
```

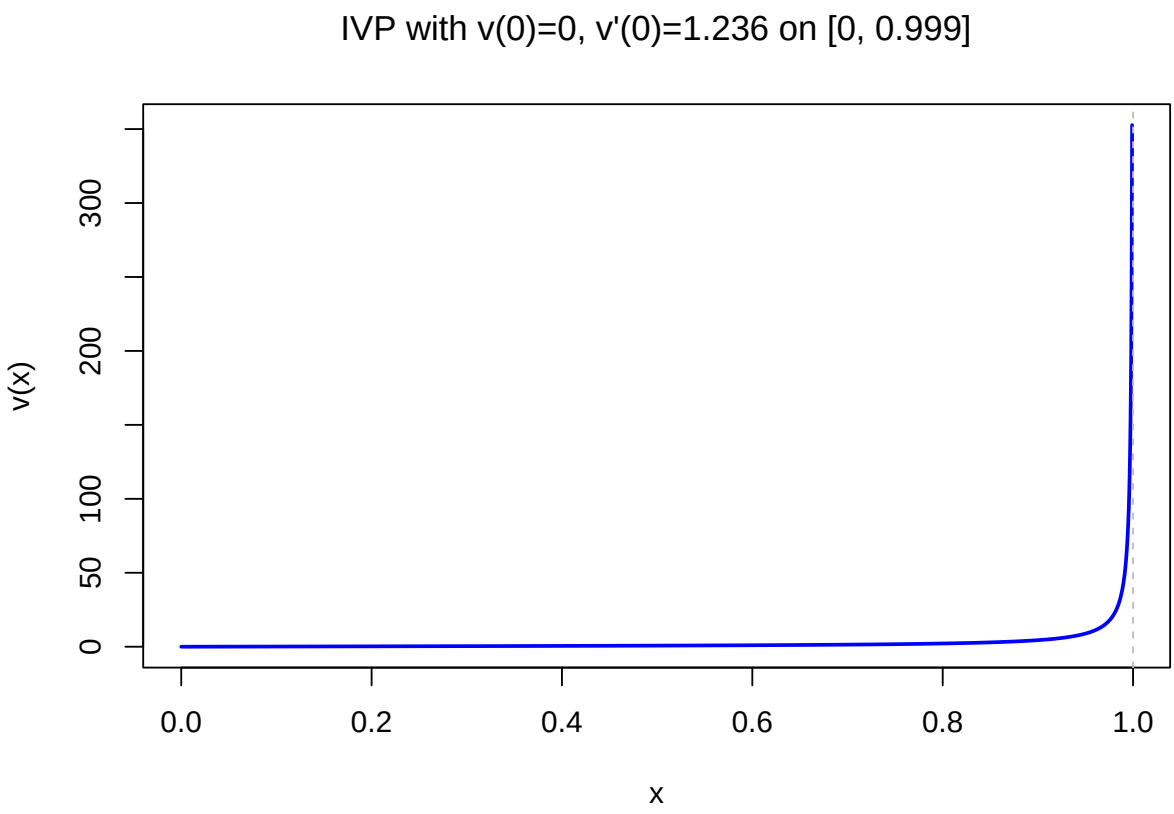
```
##   beta = 3.5 : converged, alpha = 0.909079, v(1) = 3.500000
##   beta = 3.8 : converged, alpha = 0.930668, v(1) = 3.800000
##   beta = 4.0 : converged, alpha = 0.943565, v(1) = 4.000000
##   beta = 4.5 : converged, alpha = 0.971544, v(1) = 4.500000
```

Bisection finds solutions even for $\beta > \beta_c$. So the problem isn't that solutions don't exist; it's that Newton can't find them.

Why Newton fails

The hint says to look at the IVP with $v'(0) = 1.236$ on $[0, 0.999]$:

```
sol_hint <- ode(y = c(0, 1.236), times = seq(0, 0.999, length.out = 2000),
                func = ydot_r, parms = NULL, method = "ode45")
plot(sol_hint[, 1], sol_hint[, 2], type = "l", lwd = 2, col = "blue",
     xlab = "x", ylab = "v(x)",
     main = expression(paste("IVP with v(0)=0, v'(0)=1.236 on [0, 0.999]")))
abline(v = 1, lty = 2, col = "gray")
```



The solution shoots up steeply near $x = 1$. What's happening is that for α values producing large β, the solution develops a near-singularity:

$$v(x;\alpha) \to \infty \text{ as } x \to 1 \quad \Longrightarrow \quad w(1) = \frac{\partial v(1)}{\partial \alpha} \to \infty$$

Newton relies on $w(1)$ to compute $\Delta\alpha = -(v(1) - \beta)/w(1)$. When $w(1)$ blows up, the Newton steps become unreliable and the iteration diverges. Bisection doesn't have this problem — it only needs the sign of $v(1;\alpha) - \beta$, not its derivative. That's why bisection is more robust here.

Q4 – Invertibility of a variable-coefficient tridiagonal matrix (5 points)

We have an $n \times n$ tridiagonal matrix A with diagonal entries a_i, sub-diagonal b_i (in row i), super-diagonal c_i (in row i), satisfying:

$$|b_i| + |c_i| \leq |a_i| \;\; (i = 2, \ldots, n-1), \qquad |c_1| < |a_1|, \qquad |b_n| < |a_n|$$

Goal: Show A is invertible.

Proof by contradiction

Suppose A is not invertible, so there's some nonzero X with $AX = 0$. Normalize so $\|X\|_\infty = 1$. We'll show this leads to a contradiction.

Writing out $AX = 0$ row by row:

$$\begin{aligned} \text{Row 1:} \quad & a_1 x_1 + c_1 x_2 = 0 \\ \text{Row } i: \quad & b_i x_{i-1} + a_i x_i + c_i x_{i+1} = 0 \qquad (2 \leq i \leq n-1) \\ \text{Row } n: \quad & b_n x_{n-1} + a_n x_n = 0 \end{aligned}$$

Step 1 — show $|x_1| < 1$: From Row 1, $a_1 x_1 = -c_1 x_2$, so:

$$|a_1|\,|x_1| = |c_1|\,|x_2| \leq |c_1| \cdot 1 < |a_1| \quad \Longrightarrow \quad |x_1| < 1$$

We used $|x_2| \leq \|X\|_\infty = 1$ and the strict condition $|c_1| < |a_1|$.

Step 2 — show $|x_i| < 1 \Rightarrow |x_{i+1}| < 1$: For $1 \leq i \leq n-2$, from Row $i+1$:

$$|a_{i+1}|\,|x_{i+1}| \leq |b_{i+1}|\,|x_i| + |c_{i+1}|\,|x_{i+2}|$$

We know $|x_i| < 1$ from the inductive hypothesis, and $|x_{i+2}| \leq 1$. The key strict inequality:

$$|b_{i+1}|\,|x_i| + |c_{i+1}| < |b_{i+1}| + |c_{i+1}| \leq |a_{i+1}| \quad \Longrightarrow \quad |x_{i+1}| < 1$$

For $i = n-1$, from the last row:

$$|a_n|\,|x_n| = |b_n|\,|x_{n-1}| \leq |b_n| < |a_n| \quad \Longrightarrow \quad |x_n| < 1$$

Putting it together: By induction,

$$|x_1| < 1 \;\Rightarrow\; |x_2| < 1 \;\Rightarrow\; \cdots \;\Rightarrow\; |x_n| < 1 \quad \Longrightarrow \quad \|X\|_\infty < 1$$

But we assumed $\|X\|_\infty = 1$ — contradiction. Therefore A is invertible. ■